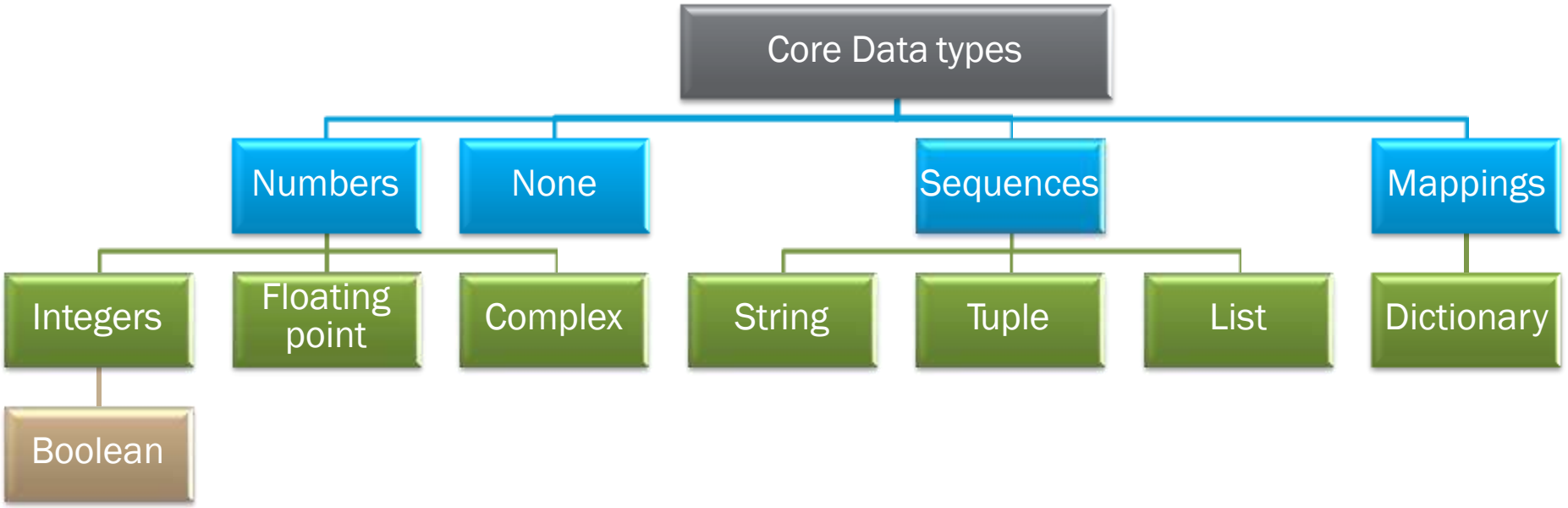
DATA TYPE SUMMARY



MUTABLE AND IMMUTABLE TYPES

Python data object can be broadly categorized into two types – mutable and immutable types. In simple words changeable/modifiable and non-modifiable types.

1. Immutable types: are those that can never change their value in place. In python following types are immutable: **integers, float, Boolean, strings, tuples**

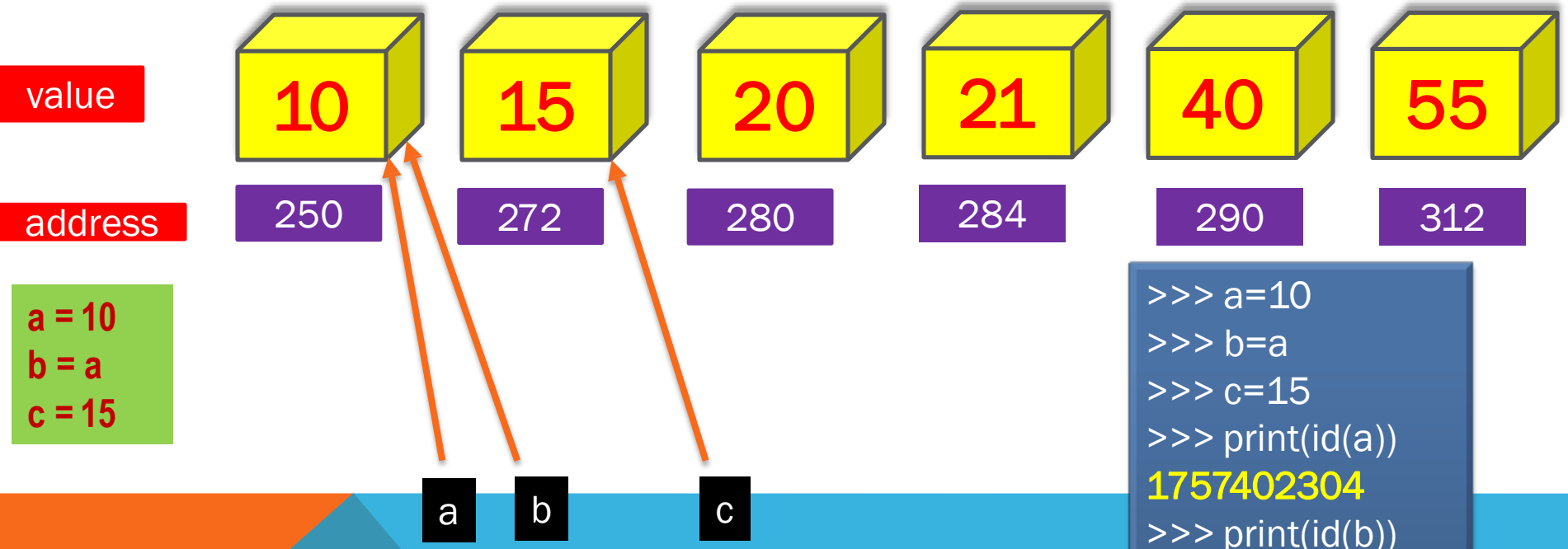
Sample Code:

```
a = 10
b = a
c = 15
a = 20
b = 40
c = b
```

From this code, you can say the value of integer a, b,c could be changed effortlessly, but this is not the case. Let us understand what was done behind the scene

IMMUTABLE TYPES

Note: In python each value in memory is assigned a memory address. So each time a new variable is pointing to that value they will be assigned the same address and no new memory allocation. Let us understand the case.



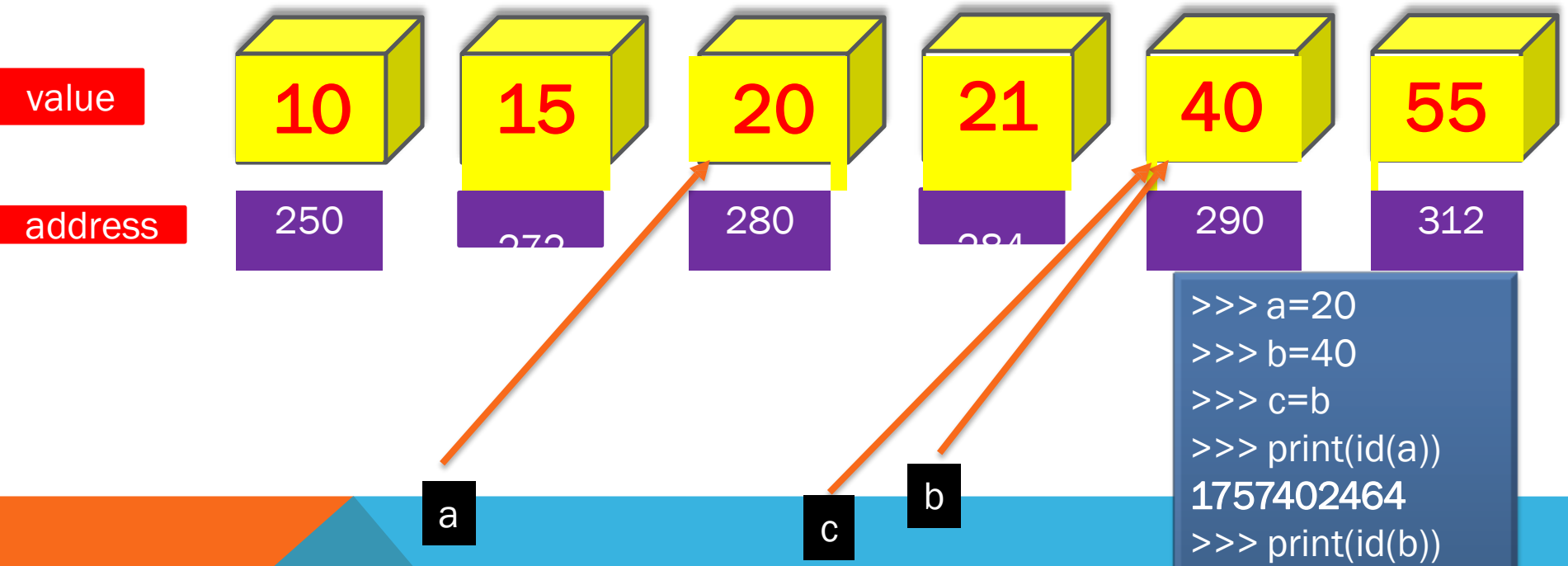

```
>>> a=10
>>> b=a
>>> c=15
>>> print(id(a))
1757402304
>>> print(id(b))
1757402304
>>> print(id(c))
1757402384
```

Python provides **id()** function to get the memory address to which value /variable is

IMMUTABLE TYPES

Now let us understand the changes done to variable a, b,c

```
a = 20
b = 40
c = b
```




```
>>> a=20
>>> b=40
>>> c=b
>>> print(id(a))
1757402464
>>> print(id(b))
1757402784
>>> print(id(c))
1757402784
```

Python provides **id()** function to get the memory address to which value /variable is referring

IMMUTABLE TYPES

From the previous code it is clear that variable names are stored references to a value-object. Each time we change the value the variable's reference memory address changes. So it will not store new value in same memory location that's why **Integer, float, Booleans, strings and tuples are immutable.**

Variables (of certain type) are NOT LIKE storage containers i.e. with fixed memory address where value changes every time. Hence they are immutable

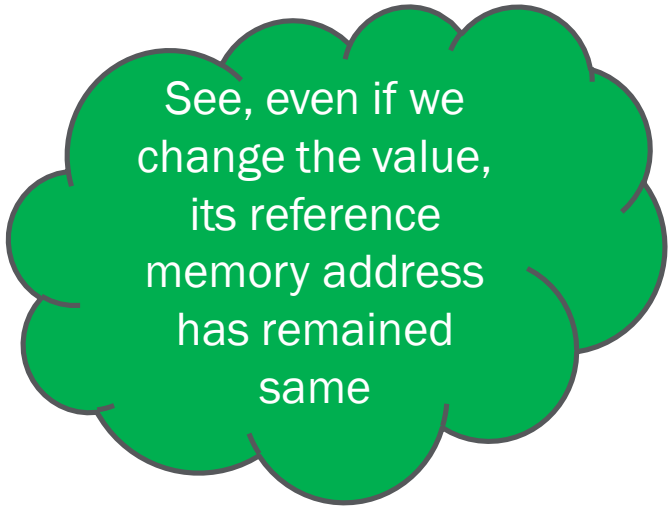
MUTABLE TYPE

Mutable means in same memory address, new value can be stored as and when it is required. Python provides following mutable types:

1. Lists
2. Dictionaries
3. Sets

Examples: (using List)

```
>>> employee=["E001","Rama","Sales",67000]
>>> print(id(employee))
71593896
>>> employee[3]=75000
>>> print(id(employee))
71593896
>>>
```



OPERATOR PRECEDENCE

Highest

Operators	Description	Associativity
()	Parenthesis	Left-to-right
**	Exponent	Right-to-left
~x	Bitwise compliment	Left-to-right
+x, -x	Positive or negative	Left-to-right
*, /, //, %	Arithmetic operator	Left-to-right
+, -	Add, Sub	Left-to-right
&	Bitwise &	Left-to-right
^	Bitwise XOR	Left-to-right
|	Bitwise OR	Left-to-right
<,<=,>,>=,<>,!=,==, is, is not	Comparison & Identity	Left-to-right
not x	Boolean Not	Left-to-right
and	Boolean AND	Left-to-right
or	Boolean OR	Left-to-right

Lowest

ASSOCIATIVITY OF OPERATORS

An expression having multiple ** operator is evaluated from right to left. i.e.

3 ** 4 ** 2 will be evaluated as 3 ** (4 ** 2) not (3 **4) ** 2

Guess the output

```
>>> 3** 4 ** 2
>>> 3 ** (4 ** 2)
>>> (3**4) ** 2
```

ASSOCIATIVITY OF OPERATORS

An expression having multiple ** operator is evaluated from right to left. i.e.

3 ** 4 ** 2 will be evaluated as 3 ** (4 ** 2) not (3 **4) ** 2

Guess the output

>>> 3** 4 ** 2 → 43046721

>>> 3 ** (4 ** 2)

>>> (3**4) ** 2

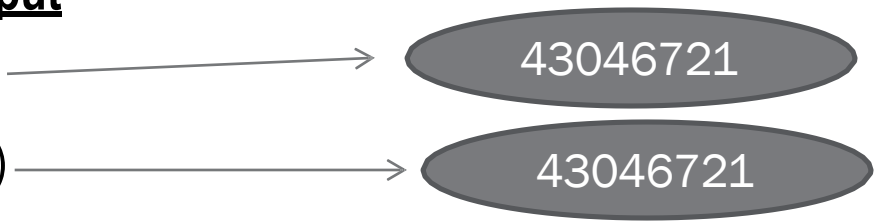
ASSOCIATIVITY OF OPERATORS

An expression having multiple ** operator is evaluated from right to left. i.e.

3 ** 4 ** 2 will be evaluated as 3 ** (4 ** 2) not (3 **4) ** 2

Guess the output

```
>>> 3** 4 ** 2
>>> 3 ** (4 ** 2)
>>> (3**4) ** 2
```



ASSOCIATIVITY OF OPERATORS

An expression having multiple ** operator is evaluated from right to left. i.e.

3 ** 4 ** 2 will be evaluated as 3 ** (4 ** 2) not (3 **4) ** 2

Guess the output

